ASJ Internal ATS + CRM

Director Project Brief | Prepared for project discussion | May 21, 2026

Main message: We built a private recruitment operating system for ASJ. It receives resumes, reads files, converts them into candidate profiles, matches candidates to open jobs, tracks pipeline stages, gives client visibility, and adds recruiter intelligence on top of live ATS data.

1. Executive Summary

- The project is an internal Applicant Tracking System (ATS) combined with lightweight CRM and AI recruiter intelligence. It is not a public marketing page; it is a working operations tool for recruiters.

- The system has one connected workflow: resume intake -> parsing -> candidate profile -> job matching -> pipeline movement -> recruiter brief. Every page exists to support one step of this flow.

- The current version is a local prototype using a Node.js backend, static frontend, local JSON database, uploaded file storage, dependency-light parsing, optional OCR, Cohere AI integration, and local matching fallback.

Current Project Snapshot

Metric	Current value	Why director should care
Candidates	6	Shows the prototype is storing structured candidate records, not only files.
Open jobs	5	Recruiters can map candidate supply against active demand.
Clients	3	CRM context is connected to jobs and recruitment delivery.
Resume inbox records	7	Supports website, manual, and bulk resume intake.
Pending / review resumes	1 pending, 1 need review	Prevents bad parsing from becoming bad candidate data.
Applications	7	Confirms candidate-job matching creates pipeline records.
Average match	63%	Gives a quick signal of shortlist quality.

2. Opening Script

Good morning. I will walk you through the ASJ Internal ATS prototype. The goal was to build a private recruitment operating system, not just a UI mockup. The system starts when a resume enters the inbox, reads and stores the file, converts useful text into a candidate profile, matches that candidate against open jobs, and lets the recruiter move the application through the hiring pipeline. We also added ATS Intelligence so the system can generate shortlist briefs, coverage gaps, interview plans, outreach drafts, and client updates from the live ATS data. I will explain the business need, why each page exists, how the backend works, and what should be improved before production.

3. Product Flow

Step	What happens	Why it was designed this way
1. Resume intake	Resume enters through website record, manual upload, or folder import.	Recruiters need a controlled intake queue before creating official profiles.
2. File storage	Original file is stored in data/uploads and linked in db.json.	The recruiter can always open the original document and audit where data came from.
3. Text extraction	PDF, DOC, DOCX, and images are read using local tools and fallback parsers.	Resumes are unstructured; the system must convert files into searchable text.
4. Quality check	Extraction is marked good, partial, or poor.	Poor scans should be reviewed or OCRed instead of polluting the candidate database.
5. Candidate creation	Parser extracts name, contact, role, experience, skills, tags, seniority, and summary.	This saves recruiter screening time and standardizes candidate data.
6. Matching	Candidate is scored against each open job by skills, experience, role category, and eligible role hints.	Recruiters can prioritize high-fit profiles while still seeing gaps.
7. Pipeline	Application cards move through Applied, Screened, Interview Round 1, Interview Round 2, HR Round, Selected, and Rejected.	Hiring work needs state tracking, not only candidate storage.
8. Intelligence	AI or local fallback produces recruiter briefs from the current ATS state.	AI is used for decisions and next actions, not as a generic chatbot.

4. Why Each Page Exists

Page / section	What it contains	Why this feature exists	How it works
Dashboard	KPIs, hiring pipeline bars, recent activity, top matching profiles, inbox health.	A director or recruiter needs the health of hiring in the first 30 seconds.	Frontend calls /api/dashboard and /api/recommendations, then renders counts and ranked matches.
AI strip	A page-level recommendation and Generate Page Brief button.	Each page should explain the most useful next action without opening the full AI page.	Uses current view name plus live ATS data through /api/ai-insight.
Resume Inbox	Pending/processed resume cards, preview, open file, parser and extraction quality.	Creates a staging area so website resumes are reviewed before becoming official candidates.	Reads websiteResumes from db.json and displays status, parser, preview text, and file link.
Upload Resume Files	Multiple files, job selection, optional corrected text.	Recruiters often receive resumes by email or local files, not only website forms.	Posts multipart data to /api/upload-resume; server validates type/size and stores the file.
Bulk Import Folder	Local folder path, job selection, import result.	Existing resume archives can be imported quickly without uploading one by one.	Backend recursively scans allowed file types and creates inbox records.
Resume Preview	File preview iframe for PDFs/images and parsed text next to it.	Recruiters can compare parsed text against the original file before import.	Browser embeds supported files and formats parsed sections in the UI.
Candidates	Boolean search bench, filters, candidate table, preview, delete.	Recruiters need practical search, filtering, resume viewing, and database cleanup.	Boolean parser supports AND, OR, NOT, brackets, quotes, skill/location/experience filters.
Jobs	Create/edit job, client, department, location, type, status, skills, description, top match.	Open roles are the demand side of recruitment; matching requires clear job skill requirements.	POST /api/jobs creates or updates jobs, then refreshes applications and match scores.
Pipeline	Kanban columns for all recruitment stages and move buttons.	Recruitment is a workflow; a candidate must be tracked from application to outcome.	PATCH /api/applications/:id updates stage and dashboard counts update after reload.
System Connections	Parser, file storage, OCR, ATS Intelligence status plus client cards.	Director can see what is live, configured, missing, or ready before production.	GET /api/system-status checks upload folder, OCR binary, AI key, and parser readiness.
ATS Intelligence	Shortlist Brief, Coverage Gaps, Interview Plan, Client Update, Outreach, custom prompt.	Makes AI operational: it explains candidates, risks, gaps, and next actions.	POST /api/ai-insight sends live ATS context to Cohere if configured; otherwise local ranking is used.
Roadmap	MVP, ATS workflow, advanced AI, integrations, user roles.	Shows the director how the prototype can become an internal product and later SaaS.	Static frontend roadmap aligned to implementation phases and role permissions.

5. Technical Architecture

Layer	Technology / file	Responsibility	Why this choice
Frontend	public/index.html, public/styles.css, public/app.js	Single-page UI, navigation, tables, cards, uploads, previews, AI output.	Fast local prototype with no build step; easy to demo and modify.
Backend	backend/server.js	HTTP server, static hosting, API routes, upload handling, parser, matching, AI calls.	Keeps the prototype dependency-light and transparent for review.
Database	data/db.json	Stores users, clients, jobs, candidates, applications, website resumes, activities.	Simple persistent storage for prototype. Production can move to PostgreSQL.
File storage	data/uploads	Stores original uploaded resumes.	Preserves the source file so candidate data is auditable.
Parser	pdftotext, textutil, tesseract when available, plus JS fallbacks.	Extracts readable text from PDFs, Word docs, and image resumes.	Works locally without forcing paid services during prototype.
AI	Cohere chat API with local fallback	Resume enrichment and recruiter intelligence briefs.	The app still works if the external key is missing.
Matching engine	scoreMatch()	Skill match, skill gaps, experience bonus, role bonus, recommendation label.	Recruiters need explainable scoring, not a black box.

Backend APIs To Know

API	Method	Purpose	Answer if asked why
/api/all	GET	Returns hydrated app data.	The frontend needs candidates connected to jobs and clients.
/api/dashboard	GET	KPIs, stage counts, activity feed.	Dashboard should be calculated from live data, not hardcoded.
/api/recommendations	GET	Top candidates, job matches, attention items.	Gives consistent local intelligence even without external AI.
/api/upload-resume	POST	Uploads and parses one resume file.	Manual intake is required for recruiter workflow.
/api/import-folder	POST	Bulk imports resume files from a folder.	Useful for migrating old resume archives.
/api/reparse-resumes	POST	Re-reads stored files and updates quality/text.	Lets recruiters improve parser results after installing OCR/tools.
/api/sync-resumes	POST	Turns readable inbox resumes into candidates and applications.	Separates intake review from official database creation.
/api/jobs	POST	Creates or updates jobs.	New roles should immediately refresh candidate matching.
/api/applications/:id	PATCH	Moves pipeline stage.	Pipeline changes should be saved, not just visual.
/api/ai-insight	POST	Generates recruiter brief.	AI reads the actual ATS state and returns operational next actions.

6. Feature Reasoning: Director Q&A

Question	Strong answer
Why build an internal ATS instead of using Excel?	Excel can list candidates, but it cannot reliably connect resumes, parsed skills, open jobs, match gaps, pipeline stages, clients, activity, and AI briefs in one workflow.
Why have a Resume Inbox before Candidates?	The inbox is a safety gate. It prevents unreadable or scanned resumes from becoming bad candidate records. Only readable/reviewed resumes are synced.
Why store the original resume file?	Recruiters and managers must verify source data. If parsing is wrong, the original file remains available for audit and manual review.
Why support PDF, Word, and images?	Recruiters receive resumes in many formats. Supporting common formats reduces manual conversion work and makes the system practical.
Why mark extraction quality?	Parsing is never perfect. Good/partial/poor quality labels make the system honest and stop overtrusting weak text extraction.
Why add Boolean search?	Recruiters already use Boolean logic in job portals. Adding AND/OR/NOT, brackets, and missing keyword feedback makes the tool familiar and testable.
Why show missing skills?	A raw score is not enough. Missing skills explain why a candidate is not selected and what to verify during screening.
Why use a Kanban pipeline?	Recruitment is stage based. A Kanban board makes progress, bottlenecks, and next actions visible.
Why include CRM clients?	ASJ recruitment is client-driven. Jobs belong to clients, and recruiters need contact and context while filling roles.
Why use AI if local ranking exists?	Local ranking gives deterministic fallback. AI adds explanation, summaries, interview plans, and communication drafts from the same data.
Why not make AI a chatbot?	A generic chatbot is vague. ATS Intelligence is constrained to recruiter tasks: shortlist, coverage gap, interview plan, outreach, and client update.
Why a local JSON database?	For a prototype, JSON is simple, visible, and easy to demo. Production should replace it with PostgreSQL and object storage.

7. Demo Script By Page

Dashboard

Start here. Say: This is the recruitment command view. It shows candidates, open jobs, pending resumes, interviews, offers, average match, top profiles, inbox health, and recent changes.

Resume Inbox

Say: This is the intake queue. Resumes from the website, upload, or folder import land here first. We can preview the original file and parsed text before importing.

Parse and Import

Say: This button converts readable pending resumes into structured candidate profiles and creates job applications based on match score.

Candidates

Say: Here the recruiter searches the database. The Boolean test bench shows matched and missing terms so the recruiter understands search quality.

Jobs

Say: Jobs define demand. Each job has required skills and a top candidate match, so the recruiter immediately sees supply fit.

Pipeline

Say: This is where recruitment execution happens. Applications move stage by stage and the dashboard reflects the current state.

System Connections

Say: This page proves the running backend status: parser, storage, OCR availability, and AI configuration.

ATS Intelligence

Say: This converts live ATS data into recruiter decisions such as shortlist briefs, coverage gaps, interview plans, outreach, and client updates.

Roadmap

Say: This shows how the prototype can mature: MVP, workflow, advanced AI, integrations, then role-based internal rollout and SaaS readiness.

8. Matching Logic

- Candidate skills are extracted from resume text by comparing against a known skill list. Jobs define required skills in the job creation form.

- scoreMatch() compares required skills with candidate skills, calculates matched skills and gaps, adds an experience bonus, adds role/category fit, then caps the score at 98.

- Recommendation labels are easy to explain: 85+ is top candidate, 70-84 is eligible, 55-69 is possible fit, below 55 is not recommended.

- This approach is intentionally explainable. A recruiter can see both the score and the reason: matched skills, missing skills, seniority, and role fit.

Current Strong Matches

Candidate	Best matched job	Score	Recommendation
Daniel Kim	Cloud DevOps Engineer	98%	matched
Ananya Rao	Data Analyst - Healthcare	91%	matched
Monish S	ML Engineer	88%	top candidate
Priya Narayan	Senior Full Stack Developer	84%	eligible

9. Production Improvements

Area	Next improvement	Why
Database	Move from db.json to PostgreSQL with migrations.	Concurrency, backups, audit, reporting, and multi-user safety.
Authentication	Add login, role-based access, and audit logs.	Admin, business, and recruiter permissions should be separated.
Storage	Move uploads to S3 or secure object storage.	Large file scale, access control, and durability.
Parsing	Use production document parsers for PDF/DOCX and OCR.	Higher accuracy for scanned resumes and complex layouts.
AI	Add prompt templates, model logs, confidence, and review controls.	Keeps AI useful, explainable, and auditable.
Integrations	Connect website form, email inbox, job boards, calendar, and notifications.	Makes ATS part of daily recruitment operations.
Security	PII encryption, malware scanning, access logs, retention policy.	Resumes contain sensitive personal data.
Reporting	Time-to-hire, source quality, client SLA, recruiter workload.	Gives management measurable recruitment performance.

10. Closing Script

To close: This prototype demonstrates the complete foundation of an internal ASJ recruitment system. It handles resume intake, parsing, candidate management, job creation, matching, pipeline movement, client context, system status, and AI recruiter intelligence. The important point is that each feature is connected to the hiring workflow. The next step is to decide whether ASJ wants to convert this prototype into a production internal tool, starting with secure database storage, authentication, better parsing, and direct website integration.

Generated from current project data at 2026-05-21 16:17.